

Docker Interview Q&A;

DevOps Engineer · Junior / Fresher · 3–5 LPA

AWS EKS · Jenkins CI/CD · GitHub Actions · Argo CD · Kubernetes · Docker · Trivy · Prometheus · Grafana · Terraform · Amazon ECR ·
Bash · Python

15 Topic Areas — Fundamentals → Architecture → Images → Dockerfile

Commands & Networking — Key commands, network types, port mapping, troubleshooting

Volumes & Compose — Persistence, bind mounts, multi-service Compose setup

Security & CI/CD — Trivy scanning, Jenkins pipeline, ECR, image tagging strategy

K8s / EKS Integration — ImagePullPolicy, ECR auth, CrashLoopBackOff, rollbacks

Scenario Q&A; — Real troubleshooting scenarios with step-by-step answers

Sanket Ajay Chopade

github.com/Sanket006 · Pune, India

Docker Interview Q&A; — DevOps Engineer (3–5 LPA)

Covers all major Docker topics for fresher/junior DevOps roles with AWS, Kubernetes, Jenkins, and CI/CD focus.

Table of Contents

- [Docker Fundamentals](#1-docker-fundamentals)
- [Docker Architecture](#2-docker-architecture)
- [Docker Images](#3-docker-images)
- [Dockerfile](#4-dockerfile)
- [Docker Commands](#5-docker-commands)
- [Docker Networking](#6-docker-networking)
- [Docker Volumes & Storage](#7-docker-volumes--storage)
- [Docker Compose](#8-docker-compose)
- [Docker Security](#9-docker-security)
- [Docker + CI/CD (Jenkins / GitHub Actions)](#10-docker--cicd)
- [Docker + Kubernetes / EKS](#11-docker--kubernetes--eks)
- [Docker Registry & ECR](#12-docker-registry--ecr)
- [Docker Troubleshooting](#13-docker-troubleshooting)
- [Docker Best Practices](#14-docker-best-practices)
- [Experience-Based Questions](#15-experience-based-questions)

1. Docker Fundamentals

What is Docker?

- Docker is an open-source platform for building, shipping, and running applications inside **containers**
- Containers package an application with all its dependencies, libraries, and configuration into a single portable unit
- Docker ensures "it works on my machine" becomes "it works everywhere"

Why do organizations use Docker?

- **Consistency** — same environment from dev to production
- **Portability** — runs on any OS or cloud that has Docker installed
- **Speed** — containers start in milliseconds vs minutes for VMs
- **Resource efficiency** — containers share the host OS kernel, no hypervisor overhead
- **Scalability** — easy to spin up or tear down multiple instances

What problem does Docker solve?

- Eliminates the "works on my machine" problem
- Removes dependency conflicts between applications on the same host
- Simplifies deployment — ship the container, not just the code
- Enables microservices by isolating each service in its own container

Docker vs Virtual Machines

Feature	Docker (Container)	Virtual Machine
OS	Shares host OS kernel	Each VM has its own OS
Size	MBs	GBs

Feature	Docker (Container)	Virtual Machine
Startup time	Milliseconds	Minutes
Isolation	Process-level	Full hardware-level
Overhead	Low	High
Use case	App packaging, microservices	Full OS isolation

What are Docker Images?

- A Docker image is a **read-only, layered template** used to create containers
- Built from a `Dockerfile`; each instruction creates a new layer
- Stored in a registry (Docker Hub, ECR)

What are Docker Containers?

- A container is a **running instance of a Docker image**
- Adds a thin writable layer on top of the read-only image layers
- Isolated from other containers using Linux namespaces and cgroups

What is Docker Engine?

- The core runtime that builds and runs containers
- Consists of: **Docker Daemon** (server), **REST API**, and **Docker CLI** (client)

What is Docker Hub?

- The default public **container registry** maintained by Docker Inc.
- Hosts official images (nginx, mysql, python, etc.) and user-uploaded images
- Free for public images; private repos require a paid plan

What happens when you run `docker run`?

Docker CLI sends request to Docker Daemon via REST API
 Daemon checks if the image exists locally
 If not, it pulls the image from the registry
 Daemon creates a container from the image
 Allocates a writable layer, network, and filesystem
 Starts the container process

2. Docker Architecture

Docker Architecture Overview

```

Docker CLI (Client)
|
| REST API
v
Docker Daemon (dockerd)
|
|-- containerd (container lifecycle manager)
|
|-- runc (OCI-compliant container runtime)
  
```

- **Docker Client** — CLI tool (`docker` commands); sends API calls to the daemon
- **Docker Daemon** (`dockerd`) — listens for API requests, manages images/containers/networks/volumes

- **containerd** — manages container lifecycle (start, stop, pull images)
- **runc** — low-level OCI runtime that actually creates the container process

How does Docker Client communicate with Daemon?

- Via a **Unix socket** (`/var/run/docker.sock`) on the same host by default
- Can also communicate over **TCP** for remote Docker daemons (with TLS)

What is containerd?

- An industry-standard container runtime, part of CNCF
- Handles image pull, container execution, and storage
- Docker Daemon delegates actual container operations to containerd
- Also used directly by Kubernetes (CRI-containerd)

What is OCI (Open Container Initiative)?

- A standard specification for container image format and runtime
- Ensures containers built by Docker can run on any OCI-compliant runtime
- Two specs: **image-spec** (image format) and **runtime-spec** (container execution)

Container Lifecycle

Created → Running → Paused → Running → Stopped → Deleted

- `docker create` → `docker start` → `docker pause/unpause` → `docker stop` → `docker rm`

3. Docker Images

How do you create a Docker image?

- Write a `Dockerfile` with build instructions
- Run `docker build -t image-name:tag .`
- The `.` refers to the **build context** (directory sent to the daemon)

What are Image Layers?

- Every instruction in a `Dockerfile` creates a **new read-only layer**
- Layers are stacked; each layer only stores the **diff** from the previous one
- Shared layers across images reduce disk usage and pull time
- Example: two images using the same `python:3.11` base share that base layer

How does Layer Caching work?

- Docker checks if a layer already exists in cache before rebuilding
- If an instruction and its context haven't changed → **cache hit**, layer reused
- Once any layer is invalidated, all subsequent layers are rebuilt
- Best practice: put frequently changing instructions (like `COPY . .`) **after** stable ones (like `RUN pip install`)

How do you view image history?

```
docker history image-name:tag
```

Shows each layer, its size, and the command that created it

Remove unused images

```
docker image prune          # removes dangling images
docker image prune -a       # removes all unused images
```

4. Dockerfile

Common Dockerfile Instructions

Instruction	Purpose
FROM	Sets the base image
WORKDIR	Sets the working directory inside the container
COPY	Copies files from build context into the image
ADD	Like COPY, but also supports URLs and auto-extracts tarballs
RUN	Executes commands during image build (creates a new layer)
CMD	Default command to run when container starts (can be overridden)
ENTRYPOINT	Fixed command; container always runs this (harder to override)
EXPOSE	Documents which port the app listens on (does NOT publish the port)
ENV	Sets environment variables inside the image
ARG	Build-time variables (not available at runtime)
VOLUME	Declares a mount point for external storage
USER	Sets the user the container process runs as

CMD vs ENTRYPOINT

	CMD	ENTRYPOINT
Purpose	Default arguments/command	Fixed executable for the container
Override	Easily overridden with <code>docker run image</code>	Not easily overridden; needs <code>--entrypoint</code> flag
Use case	Default behavior that can be changed	Containers meant to run like a specific command
Combined	CMD provides default args to ENTRYPOINT	ENTRYPOINT defines the binary

Example:

```
ENTRYPOINT ["python"]  
CMD ["app.py"]           # Run: python app.py – CMD args passed to ENTRYPOINT
```

COPY vs ADD

	COPY	ADD
Simple file copy	Yes	Yes
Copy from URL	No	Yes
Auto-extract <code>.tar</code>	No	Yes
Best practice	Prefer COPY — it's explicit	Use only when you need URL/tar features

What is a Multi-Stage Build?

- Uses multiple `FROM` statements in a single Dockerfile
- Earlier stages compile/build the app; the final stage copies only the artifact
- Drastically reduces final image size by leaving build tools behind

```
# Stage 1 – Build
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go build -o main .

# Stage 2 – Runtime (tiny image)
FROM alpine:3.18
COPY --from=builder /app/main .
CMD ["/main"]
```

Dockerfile Optimization (Reduce Image Size)

- Use **Alpine** or **Distroless** base images
- Use **multi-stage builds**
- Combine `RUN` commands with `&&` to reduce layers
- Add a `.dockerignore` file to exclude unnecessary files
- Remove temporary files and caches in the same `RUN` layer

Practical Dockerfile Analysis

```
FROM python:3.11          # Base image: Python 3.11 on Debian
WORKDIR /app              # Set working directory to /app
COPY . .                  # Copy all files from build context to /app
RUN pip install -r requirements.txt # Install Python dependencies
CMD ["python", "app.py"]  # Default command to run the application
```

How to optimize:

```
FROM python:3.11-slim      # Use slim variant – smaller base
WORKDIR /app
COPY requirements.txt .    # Copy only requirements first
RUN pip install --no-cache-dir -r requirements.txt # Leverage cache; skip pip cache
COPY . .                  # Copy app code after deps (better caching)
USER appuser              # Run as non-root
CMD ["python", "app.py"]
```

5. Docker Commands

Key Commands Reference

```
# Container lifecycle
docker run -d -p 8080:80 --name myapp nginx # Run container in background
```

```

docker start myapp          # Start stopped container
docker stop myapp          # Graceful stop (SIGTERM)
docker kill myapp          # Force stop (SIGKILL)
docker rm myapp            # Remove stopped container
docker rm -f myapp         # Force remove running container

# Listing and inspection
docker ps                  # Running containers
docker ps -a              # All containers including stopped
docker images              # List local images
docker inspect myapp       # Detailed JSON info about container
docker stats               # Live CPU/memory usage
docker logs myapp          # Container stdout/stderr logs
docker logs -f myapp       # Follow logs

# Exec into running container
docker exec -it myapp bash  # Interactive shell
docker exec myapp ls /app   # Run single command

# Image management
docker build -t myapp:1.0 . # Build image
docker pull nginx:latest    # Pull from registry
docker push myrepo/myapp:1.0 # Push to registry
docker rmi myapp:1.0        # Remove image
docker image prune -a       # Remove all unused images

# Copy files
docker cp myapp:/app/config.yaml ./config.yaml # From container to host
docker cp ./config.yaml myapp:/app/            # From host to container

```

stop **VS** kill

- `docker stop` sends **SIGTERM** → app can shut down gracefully → after timeout (10s) sends SIGKILL
- `docker kill` sends **SIGKILL** immediately → abrupt termination, no cleanup

run **VS** start

- `docker run` = creates a **new container** from an image and starts it
- `docker start` = starts an **existing stopped container**

6. Docker Networking

Docker Network Types

Network	Description	Use Case
bridge	Default; containers get private IPs on a virtual bridge	Single-host container communication
host	Container shares host's network stack	Performance-critical apps, avoid port mapping
overlay	Multi-host networking via VXLAN	Docker Swarm, cross-host communication
none	No network interfaces	Maximum isolation
macvlan	Container gets a MAC address on the host network	Legacy app integration

Port Mapping

- `-p 8080:80` maps **host port 8080** to **container port 80**
- Requests to `localhost:8080` are forwarded to the container on port 80
- Without `-p`, the container's port is only accessible within Docker's network

EXPOSE vs -p

- `EXPOSE 80` in Dockerfile is **documentation only** — tells users the app listens on 80; does NOT publish the port
- `-p 8080:80` in `docker run` **actually publishes** the port to the host

Container Communication on the Same Host

- On the default bridge network, containers communicate via **IP address only**
- On a **user-defined bridge** (`docker network create`), containers can also communicate by **container name** (DNS)
- Best practice: always create a named network for app containers

Troubleshooting: Container Running but App Inaccessible

Check if the port is published — `docker ps` (PORTS column)

Verify the app is actually listening — `docker exec -it netstat -tlnp`

Check container logs — `docker logs`

Check firewall rules on the host

Verify the app binds to `0.0.0.0`, not `127.0.0.1`

7. Docker Volumes & Storage

Why Are Volumes Needed?

- Container filesystems are **ephemeral** — deleted when the container is removed
- Volumes provide **persistent storage** that survives container restarts and deletion
- Also used to share data between containers

Volumes vs Bind Mounts

	Docker Volume	Bind Mount
Managed by	Docker	Host OS user
Location	<code>/var/lib/docker/volumes/</code>	Any path on host
Portability	High	Host-path dependent
Best for	Databases, stateful apps	Dev (live code reload)
Backup	<code>docker volume inspect</code> + copy	Direct host access

Key Volume Commands

```

docker volume create mydata          # Create named volume
docker run -v mydata:/app/data nginx  # Mount volume into container
docker run -v /host/path:/app/data nginx # Bind mount
docker volume inspect mydata         # Inspect volume details
docker volume ls                     # List volumes
docker volume prune                  # Remove unused volumes

```

Named vs Anonymous Volumes

- **Named** — `docker run -v mydata:/app/data` — explicitly named, reusable, easy to manage

- **Anonymous** — `docker run -v /app/data` — Docker assigns a random name; harder to track

Scenario: Persist MySQL Data

```
docker run -d \
  --name mysql-db \
  -e MYSQL_ROOT_PASSWORD=secret \
  -v mysql-data:/var/lib/mysql \   # Named volume for persistence
  mysql:8.0
```

Even if the container is deleted, `mysql-data` volume retains all data.

8. Docker Compose

What is Docker Compose?

- A tool for defining and running **multi-container applications** using a single YAML file (`docker-compose.yml`)
- Manages the full lifecycle: build, start, stop, network, volumes — with one command

Key Compose Commands

```
docker compose up -d           # Start all services in background
docker compose down           # Stop and remove containers/networks
docker compose down -v        # Also removes volumes
docker compose ps              # List service status
docker compose logs -f         # Follow all service logs
docker compose build           # Build/rebuild images
docker compose scale web=3     # Scale a service (legacy: --scale in v2+)
```

How Services Communicate in Compose

- Compose automatically creates a **shared network** for all services
- Services communicate using **service names as hostnames**
- `backend` service can reach `database` service at `database:5432` — no IP needed

docker-compose.yml Example (Frontend + Backend + Database)

```
version: "3.8"

services:
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend
    environment:
      - REACT_APP_API_URL=http://backend:5000

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    depends_on:
      - database
    environment:
      - DB_HOST=database
      - DB_PORT=5432

  database:
    image: postgres:15
    volumes:
      - db-data:/var/lib/postgresql/data
    environment:
```

```
- POSTGRES_PASSWORD=secret

volumes:
  db-data:
```

Environment Variables in Compose

- Define inline under `environment`:
- Or use an `.env` file in the same directory — Compose auto-loads it
- Reference with `${VARIABLE_NAME}` in the compose file

9. Docker Security

How to Secure Docker Containers

- Run containers as **non-root users** (`USER appuser` in Dockerfile)
- Use **read-only filesystems** where possible (`--read-only` flag)
- Drop unnecessary Linux capabilities (`--cap-drop ALL`)
- Use **minimal base images** (Alpine, Distroless) to reduce attack surface
- Never store secrets as environment variables or bake them into images
- Scan images for vulnerabilities before deployment (Trivy, Snyk)
- Enable **Docker Content Trust** to verify image signatures
- Restrict container resource usage with `--memory` and `--cpus` limits

Why Not Run Containers as Root?

- If a container process is compromised, root access = host-level risk
- Root inside container maps to root on the host unless user namespacing is configured
- Best practice: create a dedicated user in Dockerfile

```
RUN adduser --disabled-password appuser
USER appuser
```

What is Trivy?

- An open-source **vulnerability scanner** for container images, filesystems, and IaC
- Scans OS packages and language-specific dependencies against CVE databases
- Used in CI/CD pipelines to gate deployments on critical vulnerabilities

```
trivy image myapp:1.0
trivy image --severity CRITICAL,HIGH myapp:1.0
```

Managing Secrets in Docker

- **Never** hardcode secrets in Dockerfiles or environment variables in images
- Use **Docker Secrets** (Swarm) or **Kubernetes Secrets** with ESO/AWS Secrets Manager
- In dev, use `.env` files excluded via `.gitignore`
- In CI/CD, inject secrets as masked environment variables from the pipeline secret store

Scenario: Critical Vulnerability Found in Scan

Do not deploy — fail the pipeline at the scan stage

Identify the affected package and version

Update the base image or the specific package

Rebuild and re-scan until vulnerabilities are resolved

If a zero-day with no fix: apply mitigating controls (WAF, network policy)

10. Docker + CI/CD

Jenkins Pipeline with Docker

```

pipeline {
  agent any
  environment {
    IMAGE_NAME = "myapp"
    IMAGE_TAG  = "${BUILD_NUMBER}"
    ECR_REPO   = "123456789.dkr.ecr.ap-south-1.amazonaws.com"
  }
  stages {
    stage('Git Checkout') {
      steps { checkout scm }
    }
    stage('Docker Build') {
      steps {
        sh "docker build -t ${IMAGE_NAME}:${IMAGE_TAG} ."
      }
    }
    stage('Trivy Scan') {
      steps {
        sh "trivy image --exit-code 1 --severity CRITICAL ${IMAGE_NAME}:${IMAGE_TAG}"
      }
    }
    stage('Push to ECR') {
      steps {
        sh "aws ecr get-login-password | docker login --username AWS --password-stdin ${ECR_REPO}"
        sh "docker tag ${IMAGE_NAME}:${IMAGE_TAG} ${ECR_REPO}/${IMAGE_NAME}:${IMAGE_TAG}"
        sh "docker push ${ECR_REPO}/${IMAGE_NAME}:${IMAGE_TAG}"
      }
    }
    stage('Deploy') {
      steps {
        sh "kubectl set image deployment/${IMAGE_NAME} app=${ECR_REPO}/${IMAGE_NAME}:${IMAGE_TAG}"
      }
    }
  }
}

```

Explanation of each stage:

- **Git Checkout** — Pulls latest source code
- **Docker Build** — Builds image tagged with Jenkins `BUILD_NUMBER` for traceability
- **Trivy Scan** — Scans for CRITICAL vulnerabilities; `--exit-code 1` fails the pipeline if found
- **Push to ECR** — Authenticates to ECR using AWS CLI, tags, and pushes the image
- **Deploy** — Updates the Kubernetes deployment with the new image using rolling update

Image Tagging Strategy

- Never use `latest` in production — it's ambiguous and breaks rollback
- Tag with: `git commit SHA`, `BUILD_NUMBER`, or semantic version (`v1.2.3`)
- Promote tags across environments: `myapp:42-dev` → `myapp:42-staging` → `myapp:42-prod`

Rollback a Bad Image

```

# Kubernetes rolling rollback
kubectl rollout undo deployment/myapp

# Roll back to specific revision
kubectl rollout history deployment/myapp
kubectl rollout undo deployment/myapp --to-revision=3

```

11. Docker + Kubernetes / EKS

Why Do Kubernetes Pods Use Container Images?

- Kubernetes runs containers, not source code
- Every pod spec references an image from a registry
- Images are the **deployment artifact** — immutable and versioned

ImagePullPolicy Options

Policy	Behavior
Always	Pulls the image every time a pod starts
IfNotPresent	Pulls only if the image is not on the node
Never	Never pulls; image must already exist on the node

- If tag is `latest`, Kubernetes defaults to `Always`
- If tag is a specific version, defaults to `IfNotPresent`

Using ECR Images in EKS

Create an ECR repository

Attach an IAM role to EKS node group (or use IRSA) with `ecr:GetAuthorizationToken` and `ecr:BatchGetImage` permissions

Reference the full ECR URI in pod spec: `123456789.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0`

For private ECR: EKS nodes auto-authenticate via IAM role — no `imagePullSecret` needed

Docker Container vs Kubernetes Pod

	Docker Container	Kubernetes Pod
Unit	Single container	One or more containers sharing network/storage
Scheduling	Manual	Kubernetes scheduler
Self-healing	No	Yes (pod restart policy)
Networking	Docker networking	Kubernetes networking (CNI)
Use case	Local dev, CI builds	Production workloads

Troubleshoot: CrashLoopBackOff

```
kubectl describe pod <pod-name>      # Check Events section for errors
kubectl logs <pod-name>               # Check application logs
kubectl logs <pod-name> --previous    # Logs from previous crash

# Common causes:
# - Application error on startup (check app logs)
# - Wrong CMD/ENTRYPOINT in Dockerfile
# - Missing environment variable or config
# - Failed liveness probe
# - Out of memory (OOMKilled)
```

Troubleshoot: Image Pull Failure

```
kubectl describe pod <pod-name>      # Look for: ImagePullBackOff, ErrImagePull
```

```
# Common causes and fixes:
# 1. Wrong image name/tag → correct the image reference in deployment
# 2. Private registry, no pull secret → add imagePullSecret or fix IAM role
# 3. ECR auth expired → ensure node IAM role has ECR permissions
# 4. Network issue on node → check security groups, VPC endpoints for ECR
```

12. Docker Registry & ECR

Docker Hub vs Amazon ECR

	Docker Hub	Amazon ECR
Hosting	Public cloud (Docker Inc.)	AWS-managed, private by default
Access control	Limited (free tier)	IAM-based fine-grained policies
Integration	General purpose	Native with EKS, ECS, CodePipeline
Cost	Free for public; paid for private	Pay per storage and data transfer
Security	Basic scanning	Deep integration with Trivy, Inspector

Authenticate to ECR

```
aws ecr get-login-password --region ap-south-1 | \
  docker login --username AWS --password-stdin \
  123456789.dkr.ecr.ap-south-1.amazonaws.com
```

Push/Pull Images to ECR

```
# Tag the image
docker tag myapp:1.0 123456789.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0

# Push
docker push 123456789.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0

# Pull
docker pull 123456789.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0
```

The latest Tag Problem

- `latest` does not mean the most recent image — it's just a tag like any other
- If `latest` is overwritten by a bad build, rollback becomes difficult
- In production: always use immutable, specific tags (commit SHA, version, build ID)

Immutable Tagging (ECR Feature)

- ECR supports **immutable tags** — once an image is pushed with a tag, it cannot be overwritten
- Prevents accidental overwrite of production images
- Enable in ECR repository settings: `Image tag mutability` → `IMMUTABLE`

13. Docker Troubleshooting

Container Exits Immediately

- **Cause:** The main process (`CMD/ENTRYPOINT`) completes or crashes
- **Debug:** `docker logs` and `docker inspect`

- **Common fix:** Ensure the process runs in the **foreground**, not as a background daemon

Port Mapped but App Inaccessible

Verify port mapping — `docker ps`

Check the app binds to `0.0.0.0`, not `127.0.0.1`

Inspect firewall/security group rules

Test from within the container — `docker exec -it curl localhost:80`

Docker Build is Slow

- Not using layer cache — move stable instructions (pip install) before dynamic ones (COPY .)
- Large build context — add `.dockerignore`
- Downloading large packages every build — cache apt/pip in dedicated layers

Image Size Too Large

- Switch to `alpine` or `slim` base image
- Use multi-stage builds — discard build tools in final stage
- Remove cached package manager files: `apt-get clean && rm -rf /var/lib/apt/lists/*`
- Use `.dockerignore` to exclude unnecessary files

Container High CPU / Memory

```
docker stats                # Live resource usage
docker stats --no-stream    # Snapshot
docker update --memory 512m <name> # Limit memory at runtime
```

Disk Space Full Due to Docker

```
docker system df          # Show disk usage breakdown
docker system prune -af    # Remove all unused containers, images, networks
docker volume prune       # Remove unused volumes
```

Docker Daemon Not Starting

```
sudo systemctl status docker # Check service status
sudo journalctl -u docker    # View daemon logs
# Common causes: corrupted daemon.json, port conflict, insufficient disk space
```

Container Restarts Continuously (Restart Policy)

```
docker logs <container>      # Find the crash reason
docker inspect <container>    # Check RestartCount and exit code
# Fix the application error, then redeploy
```

14. Docker Best Practices

Image Optimization

- Use **Alpine** or **Distroless** base images for minimal footprint
- Use **multi-stage builds** to separate build and runtime environments
- Combine RUN commands: `RUN apt-get update && apt-get install -y pkg && rm -rf /var/lib/apt/lists/*`
- Use `.dockerignore` to exclude: `.git`, `node_modules`, `*.log`, `tests/`

Security Best Practices

- Never run containers as `root` — always set `USER`
- Don't store secrets in environment variables or image layers
- Scan every image with Trivy before pushing to a registry

- Use specific image tags, never `latest`
- Sign images with Docker Content Trust (DCT) for production
- Apply resource limits (`--memory`, `--cpus`) to prevent noisy-neighbor issues

Build Speed

- Order Dockerfile instructions from **least to most frequently changing**
- Copy `requirements.txt` / `package.json` first, install deps, then copy source code
- Use a layer caching strategy in CI (BuildKit cache mounts)

Production Checklist

- ☐ Non-root user set
- ☐ Specific image tag used
- ☐ Trivy scan passes with no CRITICAL/HIGH
- ☐ Multi-stage build for compiled languages
- ☐ Secrets managed via Secrets Manager / Vault, not in image
- ☐ `.dockerignore` in place
- ☐ Resource limits set on containers
- ☐ Health checks configured (`HEALTHCHECK` in Dockerfile)

15. Experience-Based Questions

How did you use Docker in your internship at Hisan Labs?

Answer framework:

- Used Docker to containerize microservices deployed on **EKS**
- Built Docker images inside **Jenkins pipelines** — automated build → scan → push flow
- Tagged images with `BUILD_NUMBER` and pushed to **Amazon ECR**
- Scanned images using **Trivy** before pushing; pipeline failed on CRITICAL vulnerabilities
- Used **multi-stage builds** for Go services to produce small, distroless images (project: Go Cloud-Native GitOps Delivery Pipeline)
- Kubernetes deployments referenced ECR images; Argo CD managed GitOps sync

Describe a Docker issue you faced and resolved

Example answer:

During the Go GitOps pipeline, the image size was too large (~900MB). I analyzed the layers with `docker history` and found the build stage (Go compiler + dependencies) was being included in the final image. I introduced a multi-stage build — the first stage compiled the binary using `golang:1.21`, and the second stage used `gcr.io/distroless/static` to copy only the binary. Final image size dropped to ~12MB, which significantly reduced ECR storage costs and pull time on EKS nodes.

How did Docker integrate with Jenkins?

- Jenkins agent ran `docker build` to build the image
- Used `docker tag` with `${BUILD_NUMBER}` for versioning
- Jenkins credentials store held ECR auth token
- After Trivy scan, Jenkins ran `docker push` to ECR
- Used `docker rmi` at the end of the stage to clean up local images on the agent

How did you scan images with Trivy?

```
sh "trivy image --exit-code 1 --severity CRITICAL,HIGH ${IMAGE}:${TAG}"
```

- `--exit-code 1` makes the stage fail if vulnerabilities are found

- Results were archived in Jenkins as HTML reports using `--format template`

Scenario Quick Reference

Scenario	First Check	Resolution
App not accessible	<code>docker ps</code> (ports), container logs	Port mapping, app binding to 0.0.0.0
CrashLoopBackOff	<code>kubectl logs --previous</code>	App crash, wrong CMD, OOMKilled
Image pull failure	<code>kubectl describe pod</code> (Events)	IAM role, registry auth, wrong tag
Build slow	Layer order, <code>.dockerignore</code>	Reorder instructions, cache dependencies
Disk full	<code>docker system df</code>	<code>docker system prune -af</code>
Container exits	<code>docker logs</code> , process check	Foreground process, app error

Generated for Sanket Ajay Chopade — Junior DevOps / SRE Interview Preparation

Stack: AWS EKS · Jenkins · GitHub Actions · Argo CD · Terraform · Kubernetes · Docker · Trivy · Prometheus · Grafana